LAB SHEET 02: DYNAMIC INTERFACE STYLING & SCRIPTING INTERACTIVITY

Task 2.1: Formulate a clean dark-mode landing layout implementing custom styling overrides via CSS variables and an interactive toggler method inside vanilla JavaScript.

Answer 2.1 — Dark Mode Landing Page.

Aim: To create a dark-mode landing page using CSS variables and JavaScript toggle functionality.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dark Mode Website</title>
  <style>
    :root {
      --bg-color: #ffffff;
      --text-color: #222222;
      --button-color: #333333;
    }
    body.dark {
      --bg-color: #222222;
      --text-color: #ffffff;
      --button-color: #ffffff;
    }
    body {
 background-color: var(--bg-color);
color: var(--text-color);
font-family: Arial, sans-serif;
text-align: center;
padding: 50px;
transition: 0.3s;
```

```
    }
button {
 padding: 10px 20px;
 cursor: pointer;
background-color: var(--button-color);
color: var(--bg-color);
border: none;
border-radius: 5px;
      }
  </style>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>
    This is a simple landing page with a dark mode option.
  </p>
  <button onclick="toggleMode()">Toggle Dark Mode</button>

  <script>
 FunctiontoggleMode(){
document.body.classList.toggle("dark");
    }
  </script>
</body>
</html>
```

Task 2.2: Build a responsive web photo grid layout using CSS Flexbox rules that wraps fluidly across mobile viewports, including hovering transform animations.

Answer 2.2 — Responsive Photo Grid Using Flexbox

Aim: To create a responsive photo grid using CSS Flexbox with hover transform animations.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Photo Grid</title>
   <style>
  body {
font-family: Arial, sans-serif;
margin: 0;
 padding: 20px;
       }
h1 {
text-align: center;
       }
       .gallery {
display: flex;
 flex-wrap: wrap;
justify-content: center;
gap: 20px;
       }
       .gallery img {
width: 250px;
 height: 180px;
object-fit: cover;
border-radius: 10px;
transition: transform 0.3s;
       }
       .gallery img:hover {
transform: scale(1.1);
       }
```

```
      @media (max-width: 600px) {
         .gallery img {
width: 100%;
height: auto;
         }
      }
   </style>
</head>
<body>
   <h1>My Photo Gallery</h1>
   <div class="gallery">
      <img src="https://picsum.photos/300/200?1" alt="Photo 1">
      <img src="https://picsum.photos/300/200?2" alt="Photo 2">
      <img src="https://picsum.photos/300/200?3" alt="Photo 3">
      <img src="https://picsum.photos/300/200?4" alt="Photo 4">
      <img src="https://picsum.photos/300/200?5" alt="Photo 5">
<img src="https://picsum.photos/300/200?6" alt="Photo 6">
   </div>
</body>
</html>
```

Task 2.3: Code a fully client-side interactive Todo Application featuring runtime element append mechanisms and operational validation error flags.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">    <title>Todo
Application</title>
   <style>
body {
```

```
 font-family:
Arial, sans-serif;
text-align: center;
padding: 30px;

      }
input {
padding: 10px;

width: 250px;

      }

      button {

padding: 10px 15px;

 cursor: pointer;

      }

      #error {
 color: red;
margin: 10px;

      }
li {
   margin: 10px;
  list-style: none;
      }
.delete {
 margin-left: 10px;
background-color: red;
color: white;
 border: none;
  padding: 5px 10px;

      }

   </style>

</head>

<body>

    <h1>Todo Application</h1>

<intput type =“type” id =”taskinput” placeholder=”Enter a task”>

<button onclick=”addTack()”>Add Task</button>

<p id=”error”></p>

<ul id=”taskList”></ul>
```

```
<script>
function addTask() {
let input = document.getElementById("taskInput");
let task = input.value.trim();
 let error = document.getElementById("error");
 if (task === "") {
            error.textContent = "Please enter a task.";
return;
        }
        error.textContent = "";
let li = document.createElement("li");
li.textContent = task;
        let deleteButton = document.createElement("button");
deleteButton.textContent = "Delete";
deleteButton.className = "delete";
deleteButton.onclick = function() {
li.remove();
        };
        li.appendChild(deleteButton);
        document.getElementById("taskList").appendChild(li);
input.value = "";
}
   </script>
 </body>
</html>
```

Output 2.1

Welcome to My Website

This is a simple landing page with a dark mode option.

Output 2.2

My Photo Gallery

Photo 1 Photo 2 Photo 3
Photo 4 Photo 5 Photo 6

(Images appear in a responsive grid and enlarge when hovered.)

Output 2.3

Todo Application

[Enter a task]

[Add Task]

Task list appears below.

If **Add Task** is clicked without entering anything:

Please enter a task.

After entering a task:

Example Task [Delete]